Laborator #01

Cuprins

1. Obiective 2
2. Elemente teoretice 2
2.1 Mediul de dezvoltare Python 2
2.2 Editorul Visual Studio Code 2
2.3 Managementul pachetelor Python 3
2.4 Medii virtuale Python 4
2.5 Documentația internă 5
2.6 Structurarea codului 6
3. Cerințe de laborator 7
3.1 Distanța Manhattan dintre 2 vectori 7
4. Temă 7
5. Bibliografie 8
Anexă 8

1. Obiective

Obiectivele generale ale sesiunii curente:

- Stabilirea echipelor de lucru.
- Familiarizarea cu uneltele ce vor fi utilizate în cadrul laboratorului la disciplina IA, precum si a scopurilor acestuia.

2. Elemente teoretice

2.1 Mediul de dezvoltare Python

Pentru utilizarea limbajului de programare Python 3 va trebui să instalăm și să configurăm interpretorul Python:

- Site oficial [1]: https://www.python.org/downloads/
- Descărcați ultima versiune stabilă de Python 3.x (în acest caz 3.14) pentru sistemul de operare țintă (în continuare presupunem că vom utiliza sistemul de operare Windows 10).
- Rulați installer-ul și bifați opțiunea `Add Python to PATH`.
- Se recomandă să reporniți calculatorul (sau cel puțin să faceți un *logoff/logon*) pentru ca variabila `Path` a sistemului să fie inițializată. În Windows 11 ea va fi inițializată automat, dar va trebui să re-porniți terminalul pentru testarea mediului Python.
- Verificați în terminal buna funcționare a mediului Python: `python --version`.

În laboratoarele USV interpretorul Python este deja instalat. Veți verifica doar versiunea curentă a acestuia.

2.2 Editorul Visual Studio Code

În laboratoarele USV veți avea acces la IDE-urile Visual Studio Code (VSC) [2] si JetBrains PyCharm [3] pentru editarea fișierelor și proiectelor Python. Dacă vreți să instalați o versiune mai modernă a editorului VSC, o puteți face de pe site-ul https://code.visualstudio.com/Download.

VSC poate fi rulat și în modul portabil. În acest scop trebuie să descărcați versiunea *.zip* a VSC și să creați un director `data` în folderul de lucru VSC înainte de prima pornire [4]. Avantajul acestei versiuni este că puteți muta folderul de lucru între diverse stați, la nivel utilizator (nu aveți nevoie să instalați *system-wide* pachetul VSC). Actualizările vor trebui în schimb făcute manual.

La prima rulare activați terminalul VSC folosind glifele de control a layout-ului din colțul dreapta sus:

Pentru a rula un script Python din terminal, pozitionați-vă în folderul corect și rulati comanda `python <nume_script>` (Windows) sau `python3 <nume_script>` (Linux/macOS).

Creați un nou fișier cu extensia `.py`. VSC se va oferi sa instaleze pachetul de extensii Python – acceptați apoi reporniți IDE-ul.

Pentru a configura un mediu de lucru Python, vom iniția un nou folder de lucru: `File -> Open folder...`.

Nu se va lucra la *nivel de fișiere* – pentru un nou proiect veți deschide mereu un folder de lucru dedicat!

Ca alternativă puteți folosi PyCharm (puteți activa o licență gratuită de student pentru întreg ecosistemul de unelte JetBrains Studio, pentru 1 an de zile [5]).

2.3 Managementul pachetelor Python

Managerul oficial de pachete Python este `pip`. Acesta permite instalarea de pachete (module) Python care instalează diverse biblioteci care pot fi apelate din scripturile Python. Acest lucru ușurează semnificativ implementarea de funcționalități în aplicațiile Python.

`pip` este un tool CLI. Poate fi rulat din terminalul sistemului de operare sau din terminalul VSC. Comenzile cele mai folosite sunt:

- testarea versiunii: `pip --version`;
- instalarea unui pachet (cea mai nouă versiune disponibilă): `pip install <nume_pachet>`;
- instalarea unui pachet pe o versiune dată: `pip install <package_name>==<version_number>`;
- dezinstalarea unui pachet: `pip uninstall <nume_pachet>`;
- actualizarea unui pachet instalat: `pip install --upgrade <nume_pachet>`;
- listarea pachetelor disponibile în format tabular: `pip list`;
- listarea pachetelor disponibile în format *requirements*: `pip freeze`.

Pachete des utilizate în lucrul cu aplicații tehnice și științifice: `numpy` (pentru calcule matematice complexe), `pandas` (pentru manipularea de date tabelare), `matplotlib` (generare de grafice) și `seaborn` (generarea de grafice *eye-candy*, bazat pe matplotlib), `PyQT` sau `PySide` (pentru vizualizarea de imagini raster sau vectoriale, de exemplu diagrame, într-un mediu CLI; pentru construirea de GUI).

Instalarea de pachete multiple se poate face în formatul `pip install <nume_pachet1> <nume_pachet2> <nume_pachet3> ...`.

Pentru **managementul reproductibil** de pachete se poate folosi un fișier specializat pentru specificarea pachetelor necesare (și a versiunilor acestora) pentru a rula un script dat. Comanda de creare a acestui fișier este: `pip freeze > requirements.txt`.

Atenție: această comandă va transfera TOATE pachetele instalate în mediul curent Python! Aceasta include și pachete instalate incidental, pentru alte proiecte, dacă nu se practică izolarea folosind medii virtuale.

Pentru a reproduce stack-ul de pachete necesare, dintr-un fișier *requirements*: `pip install -r requirements.txt`. Dacă respectiva versiune de pachet este deja instalată, ea nu va mai fi re-instalată.

`pip` instalează toate dependințele unui pachet: dacă pachetul curent depinde de alte pachete (X și Y, de exemplu), acestea vor fi de asemenea instalate. Acest lucru poate duce la

dimensiuni ridicate ale anumitor proiecte (mai ales dacă pachetele instalate au multe dependințe).

Bibliotecile instalate prin `pip` provin de pe depozitul Python central PyPi (Python Package Index) [6]. Acolo pot fi consultate scopul pachetelor și dependințele acestora.

Atenție: deși pachetele PyPi sunt verificate de Python Org, organizația nu garantează că acestea nu sunt malițioase. Folosiți cu măsură!

2.4 Medii virtuale Python

Să presupunem că avem nevoie de 2 versiuni ale pachetului X. `pip` poate instala ambele versiuni dacă specificăm exact ce versiune dorim, apoi vom putea folosi versiunea dorita în scriptul nostru dacă o specificăm exact în directiva import. Dar dacă eventual dorim să facem o actualizare de versiune a pachetelor, acest lucru poate duce la un coșmar logistic.

De asemenea, dacă dorim să instalăm 2 versiuni incompatibile ale unui pachet (se întâmplă – de exemplu `PyQT` nu permite instalarea în același mediu a 2 versiuni diferite!) nu putem folosi același mediu pentru utilizarea concomitentă, ceea ce duce la un coșmar operațional.

Soluția este utilizarea unui mediu virtual Python.

Un mediu virtual Python reprezintă un director izolat care conține o instalare specifică a interpretorului Python și un set propriu de biblioteci externe, independente de cele instalate la nivelul sistemului de operare. Scopul principal este realizarea unei separări logice a dependențelor între proiecte diferite, prevenind astfel conflictele ce pot apărea atunci când două aplicații necesită versiuni incompatibile ale aceluiași pachet. Prin utilizarea acestui mecanism, instalările efectuate prin managerul de pachete `pip` sunt localizate strict în structura de fișiere a mediului respectiv, menținând integritatea configurației globale a limbajului.

Din punct de vedere tehnic, activarea unui mediu virtual modifică variabila de mediu `PATH` a procesului curent, astfel încât sistemul să identifice executabilul Python din folderul local în detrimentul celui global (asta înseamnă că un mediu activat "nu vede" alte pachete decât cele instalate în sesiunea activă – în *shell*-ul sau terminalul curent!). Această abordare facilitează **reproductibilitatea** software-ului, deoarece permite exportarea unei liste exacte de dependințe într-un fișier de configurare (prin `requirements.txt`), asigurând funcționarea identică a codului pe orice altă mașină gazdă. Utilizarea mediilor virtuale este considerată o practică standard în ingineria software, fiind esențială pentru gestionarea eficientă a ecosistemelor complexe de biblioteci.

Modul oficial de creare a mediilor virtual în Python este folosind utilitarul `venv`:

- crearea unui mediu virtual: `python -m venv .venv`;
- activare (Windows): `.venv\Scripts\activate`;
- activare (Linux/macOS): `source .venv/bin/activate`;
- dezactivare: `deactivate`.

Doar un mediu virtual poate fi activ la un moment dat, într-un proces de terminal. Acest lucru va fi observat prin promptul modificat!

Mediul virtual își păstrează configurațiile pe disc, într-un folder specializat. El este specific unei mașini, și **se reproduce pe alt sistem, nu se copie**!

Dacă folosiți un sistem de versionare (`git`), configurați corespunzător fișierul de ignore (`.gitignore`):

- adăugați `requirements.txt` în repo;
- nu adăugați niciodată folderele mediului virtual (de obicei `.venv` sau `.env`) în repo; de exemplu Github configurează aceste 2 opțiuni automat dacă marcați că repo-ul va fi de tip Python.

Avantaje:

- portabilitate;
- izolare;
- reproductibilitate;
- curățenie în sistem;
- profesionalism.

2.5 Documentația internă

Documentația nu este doar „pentru alții”, ci și pentru tine cel de peste 6 luni. Un `docstring` (prescurtare de la *documentation string*) este un literal de tip șir de caractere care apare ca primă instrucțiune într-o fereastră de definire a unui modul, a unei funcții, a unei clase sau a unei metode în Python.

Spre deosebire de comentariile obișnuite (cele care încep cu #), `docstring`-urile sunt reținute de interpretorul Python la *runtime* și pot fi accesate programatic. Acestea explică ce face o funcție, nu cum o face.

Cele mai importante caracteristici:

- Sintaxă: se scriu întotdeauna între ghilimele triple (" " " sau ' ' '). Acest lucru îți permite să scrii explicații pe mai multe rânduri.
- Accesibilitate: poți vedea documentația unei funcții direct din cod folosind atributul special `__doc__`[1] sau funcția *help()*.
- Suport IDE: editorul Python (ex. VSC) va afișa acest text într-un pop-up atunci când treci cu mouse-ul peste numele funcției oriunde în proiect.

În general se folosesc 2 formate de `docstring` - Google și NumPy. Formatul Google este mai curat și ușor de citit.

Exemplificare:

[1] Dublu *underscore* înainte și după cuvântul-cheie „doc”.

```python
def adunare(a: int, b: int) -> int:
    """
    Adună două numere întregi.

    Args:
        a (int): Primul număr.
        b (int): Al doilea număr.

    Returns:
        int: Suma numerelor.
    """
    return a + b
```

Dacă vreți ca VSC să genereze automat `docstring`-ul dintr-o funcție creată, instalați extensia `autoDocstring - Python Docstring Generator` scrisă de Nils Werner [7].

Avantajele utilizării `docstring`-urilor:

- Mentenanță: când te întorci la un proiect după 3 luni, `docstring`-ul îți spune instant ce parametri așteaptă o funcție complexă.
- Automatizare: există instrumente (precum *Sphinx* [8] sau *pdoc* [10]) care pot citi toate `docstring`-urile din proiect și pot genera automat un site web cu documentația tehnică a aplicației.
- Colaborare: într-o echipă, este singura metodă civilizată prin care colegii tăi pot înțelege cum să îți folosească modulele fără să citească toată logica internă a codului.

2.6 Structurarea codului

Conceptul de „dunder”

„Dunder” vine de la Double Underscore (dublă subliniere). Acesta denotă metode speciale precum `__init__`, `__str__` sau `__main__`. Ele definesc comportamente ascunse ale obiectelor Python (așa-numitele *magic methods*).

Conceptul de `__main__`

Acest bloc de cod previne rularea automată a scriptului atunci când este importat în alt fișier. Codul din interior va rula doar dacă executați fișierul direct.

De exemplu codul:

```python
if __name__ == "__main__":
    print("Rulat direct")
```

va fi executat dacă scriptul care conține acel cod (scriptul principal) este rulat direct în interpretorul Python; dacă scriptul în care rezidă este importat ca modul, codul nu se execută.

Module

Dacă aveți un folder cu mai multe fișiere *.py*, puteți defini un *pachet* (*package*), numit și *modul*. Pentru a informa interpretorul de acest lucru, veți plasa un fișier gol cu numele

`__init__.py` în folderul respectiv. Modulele pot fi importate (ca și pachete), folosind directiva `import`.

Structura:

```
Code

aplicatie/
│
├── main.py
└── util/
    ├── __init__.py
    └── calcule.py
```

Importul se face astfel:

```python
from util.calcule import functie
```

3. Cerințe de laborator

3.1 Distanța Manhattan dintre 2 vectori

Distanța Manhattan (numită și *L1 distance* sau *city-block distance*) dintre doi vectori reprezintă suma diferențelor absolute dintre componentele lor corespunzătoare. Dacă avem doi vectori $X = (X_1\ X_2\ \dots\ X_n)$ și $Y = (Y_1\ Y_2\ \dots\ Y_n)$, atunci distanța Manhattan este definită:

$$d(X,Y) = \sum_{i=1}^{n} |X_i - Y_i|$$

Intuitiv, această distanță măsoară cât „mergi” pe direcții ortogonale (orizontal și vertical), fără diagonale - exact ca într-un oraș cu străzi dispuse în grilă dreptunghiulară, precum Manhattan, de unde provine și denumirea.

Comparativ cu distanța euclidiană (*L2 distance*), distanța Manhattan nu ridică la pătrat diferențele și nu extrage radicalul, ceea ce o face mai robustă la valori extreme (așa-numitele *outliers*) în anumite aplicații de *machine learning*. Ea este frecvent utilizată în algoritmi precum KNN (K-Nearest Neighbors), în optimizare discretă și în spații cu dimensionalitate mare, unde L1 poate produce frontiere de decizie diferite față de L2.

4. Temă

A. Stabiliți echipele de lucru.

B. Livrabil (individual): creați un proiect Python folosind VSC, ce utilizează un mediu virtual și implementează calculul distanței Manhattan între 2 vectori (citiți dintr-un fișier de intrare) în mod manual cât și folosind funcția `cityblock()` din pachetul `scipy`. Structurați

funcțiile utilizate sub formă de modul și comentați-le. Proiectul va fi stocat într-un repo pe Github sau Gitlab.

5. Bibliografie

[1] Python Org, "Download Python", Python.org, 2019. https://www.python.org/downloads/

[2] Microsoft Visual Studio Code, "Visual Studio Code", Visualstudio.com, Apr. 14, 2016. https://code.visualstudio.com/Download

[3] JetBrains Studio, "PyCharm," JetBrains, 2025. https://www.jetbrains.com/pycharm/

[4] Microsoft, "Visual Studio Code," Visualstudio.com, Nov. 03, 2021. https://code.visualstudio.com/docs/editor/portable (accessed Feb. 22, 2026).

[5] JetBrains Studiu, "Free JetBrains Student Pack," JetBrains, 2025. https://www.jetbrains.com/academy/student-pack/

[6] Python Org, "PyPI · The Python Package Index," PyPI. https://pypi.org/

[7] Werner, Nils. "autoDocstring - Python Docstring Generator - Visual Studio Marketplace," @Code, 2026. https://marketplace.visualstudio.com/items?itemName=njpwerner.autodocstring (accessed Feb. 22, 2026).

[8] "Overview — Sphinx 4.0.0+ documentation," www.sphinx-doc.org. https://www.sphinx-doc.org/en/master/

[9] "pdoc – Generate API Documentation for Python Projects," pdov.dev. https://pdoc.dev/

Anexă

-